Intelligent Agents

A Prototype Design Report

Designing a Reactive Digital Forensics Agent for Automated Evidence Identification and Preservation

GROUP D

Haseeb Hassan Khan, Christopher Laudriec, Zee Mehmood, Lauretta Oghenevurie and Sabanathan Rubanathan

University of Essex Online
February 2026.

TABLE OF CONTENTS

1. **Introduction**……..……3
2. **System Requirements**……..…….3
3. **Agent Design and Architectural Rationale**……4
4. **File Identification Strategy**……..4
5. **Integrity and Data Processing**…….5
6. **Development Approach and Methodology**…….6
7. **Operational Risks and Mitigation Strategy**……...6
8. **System Diagram Designs**…….8

   Figure 1: Component Diagram……..8

   Figure 2: Sequence Diagram……..……9

   Figure 3: Forensic Agen Flowchart……..……10
9. **Conclusion**……...11
10. **References**……...12

1. INTRODUCTION

During the Fourth Industrial Revolution, the rapid expansion of digital communication and interconnected systems has increased the scale and complexity of digital forensic investigations (Mahmood, 2019). Consequently, organisations increasingly require automated systems capable of identifying, preserving and analysing large volumes of evidentiary data while maintaining strict chain-of-custody standards. Manual or ad-hoc file discovery processes introduce risks to evidentiary integrity, reproducibility and operational efficiency.

This proposal presents the design of a reactive Digital Forensics Agent that identifies relevant file types, validates artefacts using cryptographic hashing and securely stores evidentiary metadata for downstream analysis.

2. SYSTEM REQUIREMENTS

Operational viability requires a lightweight, cross-platform implementation capable of supporting heterogeneous investigative environments. Python (v3.9+) is selected due to its extensive ecosystem of libraries supporting file system analysis, cryptographic operations and data processing, enabling efficient and controlled development. The agent must execute consistently across Windows, Linux and macOS platforms to accommodate evidence derived from diverse systems (Quick and Choo, 2021).

Deployment is configured as a standalone read-only process by default, with privilege escalation restricted to authorised contexts. This minimises attack surface while preserving forensic capability. Core implementation components include Python libraries for file system traversal, cryptographic hashing and metadata persistence, supported by a lightweight SQLite database for storing evidentiary metadata and audit logs. Such lightweight forensic processing architectures are widely recommended for scalable digital investigations (Garfinkel, 2010).

3. AGENT DESIGN AND ARCHITECTURAL RATIONALE

The proposed system is modelled as a single reactive agent operating within a digital forensic environment comprising directory structures, metadata attributes and file content signatures. A scanning component functions as the sensing mechanism, generating percepts when files are detected or modified within monitored directories and enabling a perception–action cycle in which environmental stimuli trigger corresponding forensic processing actions.

These percepts are evaluated against predefined forensic rules, triggering actions including classification, cryptographic hashing, archiving and metadata logging. The agent's objective is to preserve evidentiary integrity while automating routine discovery tasks. Figure 1 illustrates the structural organisation of these components within the proposed architecture.

A reactive architecture is selected over deliberative models such as Belief–Desire–Intention (BDI) agents. Reactive systems respond directly to environmental stimuli without maintaining complex belief revision or goal deliberation processes (Russell and Norvig, 2021). In forensic workflows, procedures are typically deterministic and rule-driven, rendering extensive cognitive reasoning unnecessary. Although BDI systems provide advanced decision capabilities (Bordini et al., 2020), their additional computational overhead offers limited practical benefit in this domain.

The single-agent design prioritises reproducibility and auditability, both critical for legal admissibility (Chernyshev et al., 2021). By avoiding inter-agent coordination, the architecture reduces latency and implementation complexity (Wooldridge, 2020; Abar et al., 2021). While this simplicity may limit scalability in large-scale environments (Zhang et al., 2022), the design reflects a deliberate balance between efficiency and forensic defensibility.

4. FILE IDENTIFICATION STRATEGY

The agent implements a two-stage identification strategy combining extension-based filtering with header-level verification through magic number analysis, balancing efficiency and forensic accuracy.

In forensic environments, file extensions alone are unreliable as they can be deliberately modified to obscure true file content (Carrier and Spafford, 2020). Structural signature analysis improves classification reliability, with empirical evidence demonstrating accuracy rates exceeding 95% for common file types (Karampidis and Papadourakis, 2016). Header-level verification using file signature analysis (e.g., tools such as *python-magic*) therefore mitigates evidentiary omission caused by superficial file renaming.

Although header inspection introduces modest computational overhead compared with extension-only filtering, the design prioritises detection robustness and evidentiary completeness over marginal performance gains. In legally sensitive contexts, this trade-off ensures accurate identification of evidentiary artefacts.

5. INTEGRITY AND DATA PROCESSING

Following accurate identification, evidentiary integrity must be preserved at the point of acquisition. The agent therefore generates cryptographic hash values (SHA-256 and MD5) for each identified file to support contemporary security standards and interoperability with legacy forensic systems.

Hash functions provide deterministic verification mechanisms, enabling investigators to confirm that artefacts remain unaltered throughout their lifecycle (NIST, 2006). By recording hash values alongside associated metadata, the agent establishes a verifiable chain of custody and supports legal admissibility requirements (Casey et al., 2021). While SHA-256 provides strong collision resistance, MD5 remains relevant for comparison with historical forensic datasets.

Although generating both hash values introduces additional computational overhead, the design prioritises evidentiary robustness and cross-system validation over marginal performance gains.

6. DEVELOPMENT APPROACH AND METHODOLOGY

Implementation follows an object-oriented design to promote modularity and separation of concerns. Core capabilities including scanning, hashing, archiving, reporting and custody logging are encapsulated within discrete components, enabling targeted testing and incremental enhancement without disrupting system behaviour.

An iterative development methodology supports staged integration of directory traversal, file identification, integrity verification and reporting features, enabling progressive validation of each forensic capability (Schwaber and Sutherland, 2020). Quality assurance combines automated unit testing of individual components with integration testing across simulated forensic workflows. Containerised environments validate cross-platform compatibility on Windows, macOS and Linux without dedicated hardware (Jamshidi et al., 2020). These practices support reliability, portability and predictable operational performance prior to deployment.

7. OPERATIONAL RISKS AND MITIGATION STRATEGY

Operational deployment of a digital forensic agent introduces risks that must be managed to preserve evidentiary reliability and system robustness. A key concern is false positive classification, which may result in acquisition of irrelevant artefacts. Although extension and signature checks improve accuracy, single-method detection remains insufficient. Accordingly, multi-criteria validation combining file signatures, magic numbers and entropy analysis reduces misclassification rates in forensic contexts, strengthening evidentiary defensibility (Karampidis and Papadourakis, 2016).

Encrypted or permission-restricted files introduce additional constraints. Rather than terminating execution, the system employs a graceful degradation strategy, flagging and

logging inaccessible artefacts for subsequent review. This preserves investigative continuity while maintaining transparency.

Legal admissibility requires an unbroken chain of custody (Casey et al., 2021). An event-driven audit logging mechanism records file interactions and associated metadata, supporting traceability and mitigating evidentiary tampering risks.

At scale, directory traversal may introduce performance constraints. Generator-based breadth-first exploration ensures stable memory usage during large file system analysis (Beazley and Jones, 2013), supporting scalability without compromising stability.

8. SYSTEM DIAGRAM DESIGNS

To illustrate the proposed architecture and operational behaviour, the system is represented through complementary modelling views highlighting structural components, runtime interactions and workflow. These diagrams provide a visual representation of how the digital forensic agent operates within the investigative environment.

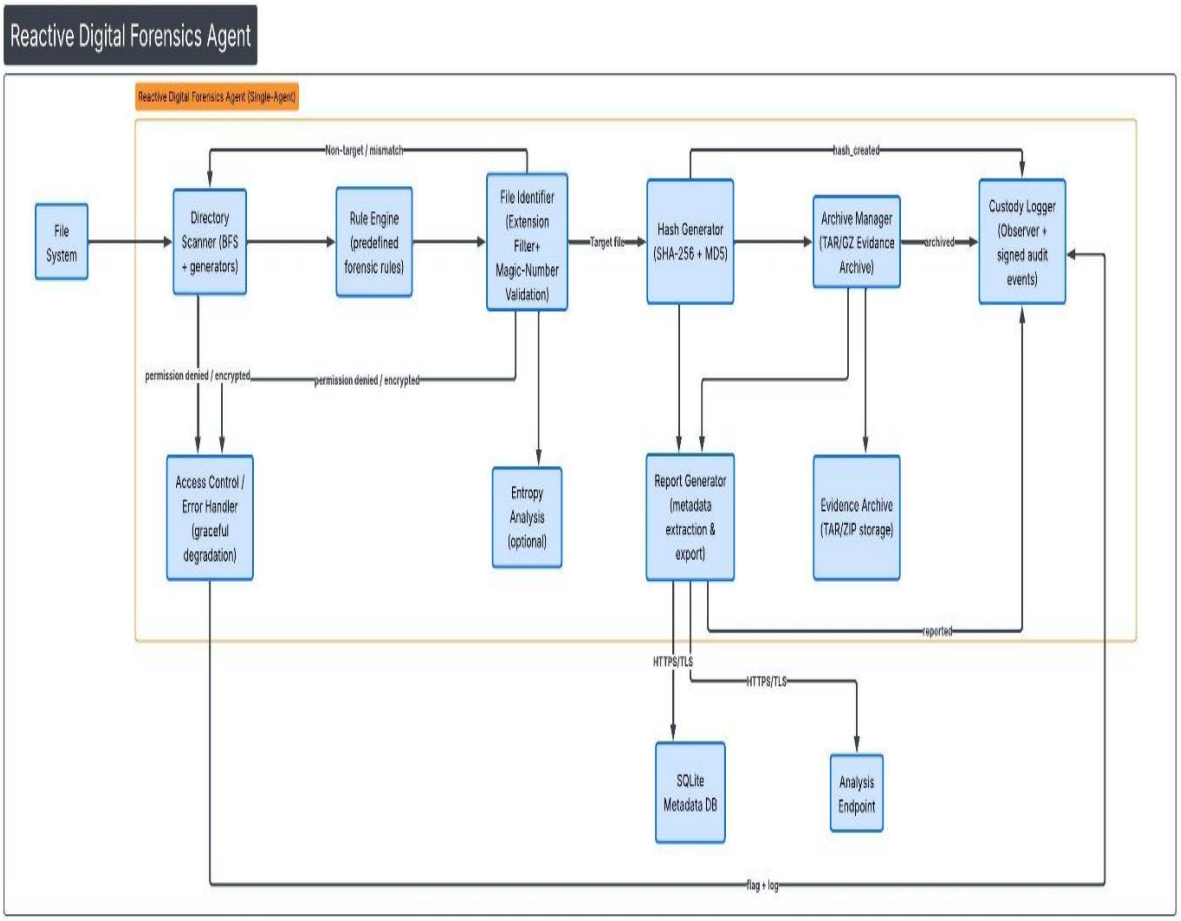

Figure 1 – Component Diagram (Structural View)

Illustrates the system architecture, including modules responsible for scanning, file identification, hashing, archiving and audit logging, and their interaction with the host file system.

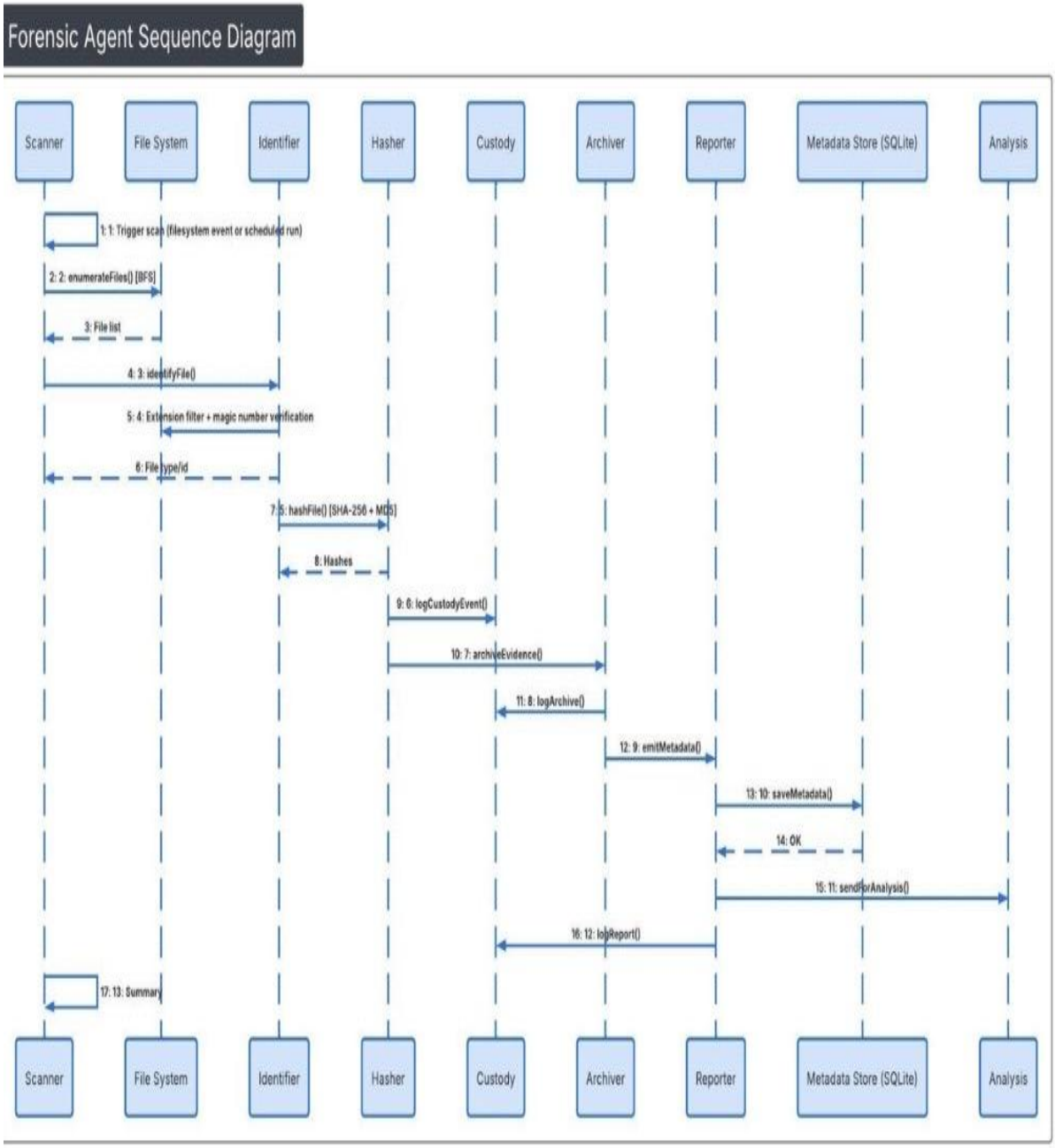

Figure 2 – Sequence Diagram (Interaction View)

Depicts the runtime interaction during a scan operation, showing how files are discovered, validated, hashed, archived and logged across system components.

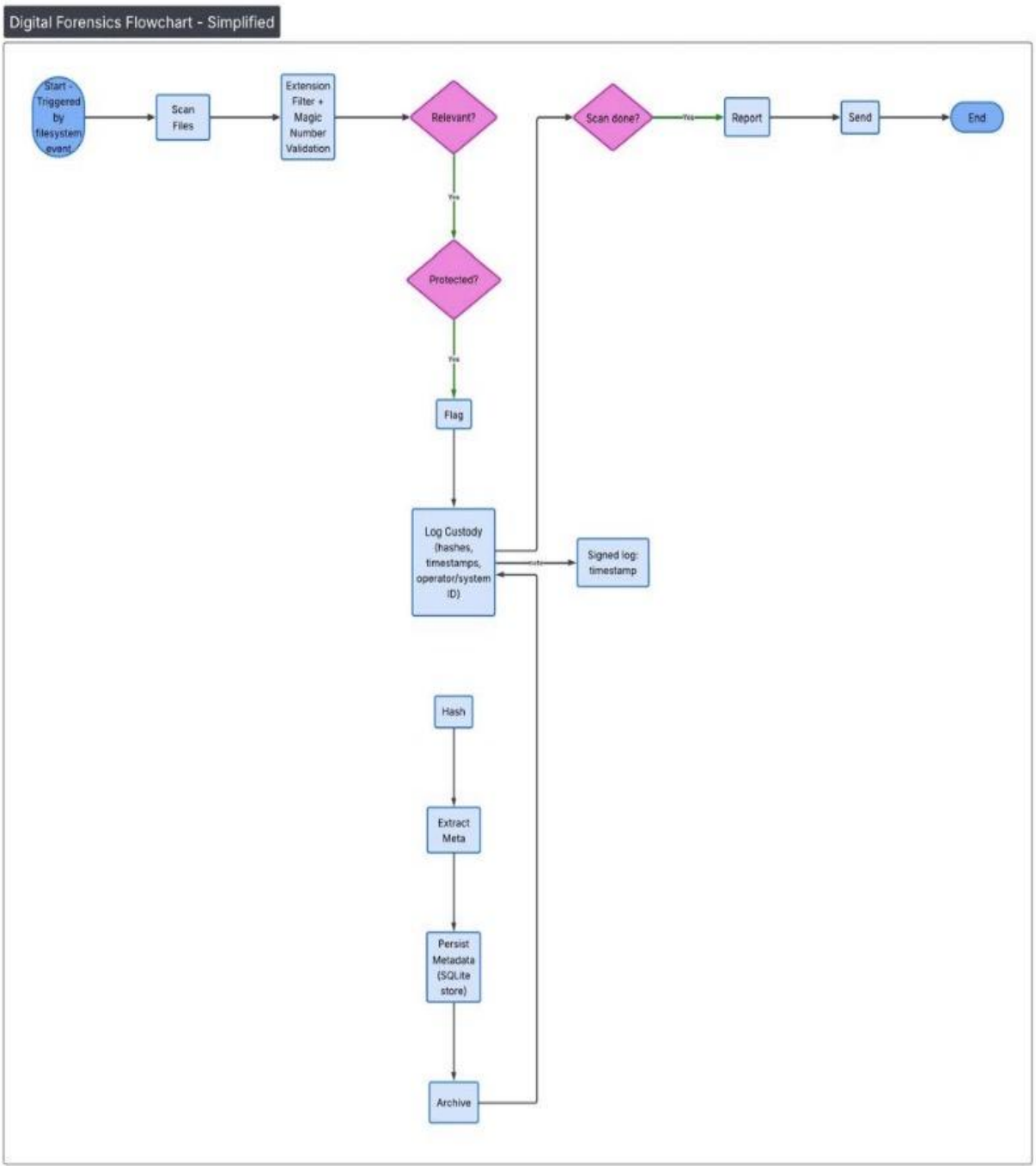

Figure 3 – Forensic Agent Flowchart (Process View)

Provides an end-to-end representation of the agent's workflow from file discovery and validation to hashing, metadata recording and reporting.

9. CONCLUSION

The proposed reactive digital forensic agent provides a structured and deterministic approach to automated file identification, integrity verification and evidentiary preservation. By integrating layered validation, cryptographic hashing and event-driven custody logging, the system enables reliable retrieval, secure processing and verifiable storage of digital artefacts.

Although scalability constraints may limit performance in large-scale environments, the design deliberately prioritises evidentiary robustness over throughput. Supported by modular architecture and disciplined development practices, the system offers a legally defensible and operationally reliable framework for contemporary digital forensic investigations.

10. REFERENCES

Abar, S., Theodoropoulos, G.K., Lemarinier, P. and O'Hare, G.M.P. (2021) 'Agent based modelling and simulation tools: A review of the state-of-art software', *Computer Science Review*, 40, 100370.

Akhtar, N. and Gupta, S. (2022) 'Performance analysis of file handling operations in Python', *International Journal of Advanced Computer Science and Applications*, 13(8), pp. 476-482. https://doi:10.14569/IJACSA.2022.0130856

Beazley, D. and Jones, B.K. (2013) *Python Cookbook*. 3rd edn. Sebastopol, CA: O'Reilly Media.

Bordini, R.H., Hübner, J.F. and Wooldridge, M. (2020) 'BDI Agent Architectures: A Survey', *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. Available at: https://www.ijcai.org/proceedings/2020/0684.pdf (Accessed: 14 February 2026).

Carrier, B. and Spafford, E.H. (2020) 'An event-based digital forensic investigation framework', *Digital Investigation: The International Journal of Digital Forensics & Incident Response*, 32, 100916.

Casey, E., Barnum, S., Griffith, R., Snyder, J., van Beek, H. and Nelson, A. (2021) 'The evolution of expressing and exchanging cyber-investigation information in a standardized form', *Forensic Science International: Digital Investigation*, 36, 301093.

Chernyshev, M., Zeadally, S., Baig, Z. and Woodward, A. (2021) 'Mobile forensics: Advances, challenges, and research opportunities', *IEEE Security & Privacy*, 19(2), pp. 80-89. https://doi:10.1109/MSEC.2020.3033896

Garfinkel, S.L. (2010) 'Digital forensics research: The next 10 years' *Digital Investigation*, 7, pp. S64-S73.

Hahsler, M. and Nagar, S. (2021) 'Design patterns for object-oriented data analysis in R', *Journal of Statistical Software*, 99(7), pp. 1-24 https://doi:10.18637/jss.v099.i07

Jamshidi, P., Pahl, C., Mendonça, N.C., Lewis, J. and Tilkov, S. (2020) 'Microservices: The journey so far and challenges ahead', *IEEE Software*, 37(3), pp. 24-35. https://doi:10.1109/MS.2020.2983475

Karampidis, K. and Papadourakis, G. (2016) 'File identification of data fragments by their binary structure', *Information Security Journal: A Global Perspective*, 25(1-3), pp. 15-24.

Kaur, R. and Kaur, A. (2022) 'Digital forensics: File carving and validation', *Forensic Science International: Digital Investigation*, 41, 301381 https://doi:10.1016/j.fsidi.2022.301381

Mahmood, Z. (2019) *The internet of things in the industrial sector*. Berlin/Heidelberg, Germany: Springer International Publishing.

Quick, D. and Choo, K.K.R. (2021) 'Digital forensic intelligence: Data subsets and Open Source Intelligence (DFINT+OSINT): A timely and cohesive mix', *Future Generation Computer Systems*, 78(1), pp. 558-567 https://doi:10.1016/j.future.2020.10.020

Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Harlow: Pearson Education.

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/ (Accessed: 10 February 2026).

Wooldridge, M. (2020) 'Temporal logics for multi-agent systems', *Artificial Intelligence*, 287, 103304. https://doi:10.1016/j.artint.2020.103304

Zhang, X., Xue, Y. and Lindqvist, J. (2022) 'Towards scalable and efficient digital forensic investigation: A unified framework', *Digital Investigation*, 41, 301441.